Day 5 - Facilitation Guide
(Functions)

Index

(1.5 hrs) ILT

I. Recap

In our last session we learned:

- **Conditional constructs:**Conditional constructs, also known as conditional statements or control structures, are fundamental programming elements that allow you to make decisions in your code based on specific conditions.
- **if Statement:**The if statement is used to execute a block of code if a specified condition is true.
- **Ladder if else:** A "ladder" of if-else statements, also known as an "if-else if-else" ladder, is a series of if, elif (short for "else if"), and else statements used to evaluate multiple conditions in a hierarchical manner. Each condition is checked one by one, and the first one that evaluates to true triggers the associated code block to execute. If none of the conditions are true, the else block (if present) is executed.
- **Nested conditions:**Nested conditions, also known as nested if statements, occur when you place one or more if, elif, or else statements inside another if, elif, or else block. This allows you to create more complex decision-making logic by evaluating conditions within conditions

In this session we are going to understand about Functions,python builtin function mathematical and user defined functions

II. Functions

In Python, a function is a reusable block of code that performs a specific task or set of tasks. Functions allow you to encapsulate logic, make your code more organized and modular, and avoid redundancy by enabling you to call the same code multiple times with different inputs.

Here's the basic syntax of a Python function:

```
def function_name(parameters):
    # Function body (code that performs a task)
    # ...
    return result  # Optional: Returns a value
```

Let's break down the components of a function:

def: This keyword is used to define a function.

function_name: Choose a descriptive name for your function to indicate its purpose. Function names should follow Python naming conventions (e.g., use lowercase with underscores for multi-word names).

parameters (also called arguments): These are optional inputs that you can pass to the function. A function can have zero or more parameters.
: (colon): It marks the beginning of the function's body.

Function body: This is where you write the code that defines the functionality of the function.

return (optional): This keyword is used to specify the value that the function should return when it's called. If a function doesn't have a return statement, it implicitly returns None.

Here's an example of a simple function that adds two numbers

```
def add_numbers(a, b):
    result = a + b
    return result
sum_result = add_numbers(5, 3)
print(sum_result)  # Output: 8
```

Functions can also have **default values** for parameters, allowing you to call the function without providing all the arguments:

```
def greet(name, greeting="Hello"):
    message = f"{greeting}, {name}!"
    return message

greet_default = greet("Alice")
greet_custom = greet("Bob", "Hi")

print(greet_default)  # Output: "Hello, Alice!"
print(greet_custom)   # Output: "Hi, Bob!"
```

Functions can be called within other functions, and you can create complex programs by organizing your code into smaller, reusable functions.

Python provides built-in functions like print(), len(), and many others, and you can also create your own custom functions to extend Python's functionality and make your code more readable and maintainable.

Advantages of Python Functions

- **Modularity:** Functions allow you to break down a complex program into smaller, more manageable pieces. Each function can represent a specific task or operation, enhancing code organization.
- **Reusability:** Once you define a function, you can reuse it multiple times throughout your code, reducing redundancy and making your code more concise.
- **Readability:** Well-named functions make your code more readable and self-documenting. A descriptive function name can indicate its purpose, making the code easier to understand.
- **Abstraction:** Functions allow you to abstract away complex operations. By encapsulating logic within functions, you provide a higher-level interface for performing tasks.
- **Parameterization:** Functions can accept arguments (parameters), making them flexible and adaptable to different inputs. This parameterization enables you to reuse functions with varying data.
- **Code Maintenance:** Functions simplify code maintenance. If you need to make changes or updates to a specific piece of functionality, you can do so within the corresponding function, reducing the risk of introducing bugs elsewhere in the code.
- **Testing and Debugging:** **UDFs(User defined Functions) can be tested independently, aiding in the debugging process. You can isolate and fix issues within specific functions, making it easier to identify and resolve problems.

****UDFs(User defined Functions)**:User-defined functions (UDFs) are reusable blocks of code created by the programmer to perform a specific task or set of tasks within a program. Functions are a fundamental concept in programming that promotes modularity, code reusability, and readability. Here are the key components and principles of user-defined functions.

Syntax for Defining a Function

```
def function_name(parameters):
  """ Docstring (optional): Description of the function. """
   # Function body: Statements to perform the task
  result = ...
  return result # Return statement (optional)
```

III . Built-in Functions

Python provides a wide range of built-in functions that are readily available for use without the need to define them. These functions cover a variety of tasks, from mathematical operations to working with data structures, input/output operations, and more. Here are some commonly used built-in functions in Python:

1. Print Function: `print()`
- Used to display output to the console.
Example:
print("Hello, World!")

2. Type Conversion Functions:
- **`int()`:** Converts a value to an integer.
- **`float()`:** Converts a value to a floating-point number.
- **`str()`:** Converts a value to a string.
Example:
int("42")
converts the string "42" to an integer.

3. Mathematical Functions:
- **`abs()`:** Returns the absolute value of a number.
- **`max()`:** Returns the maximum value in an iterable.
- **`min()`:** Returns the minimum value in an iterable.
- **`pow()`:** Raises a number to a specified power.
- **`round()`:** Rounds a floating-point number to the nearest integer.
Example:
max(4, 7, 2)
returns 7

4. String Functions:
- **`len()`:** Returns the length of a string.
- **`str.upper()`:** Converts a string to uppercase.
- **`str.lower()`:** Converts a string to lowercase.
- **`str.split()`:** Splits a string into a list of substrings.
- Example:
len("Hello")
returns 5

5. List and Tuple Functions:
- **`len()`:** Returns the number of elements in a list or tuple.
- **`list()`:** Converts an iterable (e.g., tuple) to a list.
- **`tuple()`:** Converts an iterable (e.g., list) to a tuple.
Example:
list((1, 2, 3))
converts the tuple to a list

6. Input Functions:

- **`input()`:** Reads a line of text entered by the user from the console.
Example:
name = input("Enter your name: ")

7. File Handling Functions:
- **`open()`:** Opens a file for reading or writing.
- **`read()`:** Reads the contents of a file.
- **`write()`:** Writes data to a file.
- **`close()`:** Closes a file.
Example:
file = open("example.txt", "w")

8. List Functions:
- **`append()`:** Adds an element to the end of a list.
- **`extend()`:** Adds elements from an iterable to a list.
- **`pop()`:** Removes and returns an element from a list.
- **`sort()`:** Sorts the elements of a list.
Example:
my_list.append(42)

9. Dictionary Functions:
- **`keys()`:** Returns a list of keys in a dictionary.
- **`values()`:** Returns a list of values in a dictionary.
- **`items()`:** Returns a list of key-value pairs in a dictionary.
Example:
my_dict.keys()

10. Boolean Functions:
- **`bool()`:** Converts a value to a Boolean (True or False).
- **`all()`:** Returns True if all elements in an iterable are true.
- **`any()`:** Returns True if any element in an iterable is true.
Example:
bool(42)
returns True

Note : You will learn the above mention functions uses in your upcoming session

These are just a few examples of Python's built-in functions. Python has a rich standard library with many more functions for various tasks. You can explore and use these functions as needed to simplify and optimize your code.

IV. Type Conversion Functions

Type conversion functions, also known as casting or conversion functions, are used in programming to change the data type of a value or variable from one type to another.

These functions help you ensure that data is in the correct format for various operations. The specific functions and syntax may vary depending on the programming language you are using, but the concept remains consistent.

Here are common type conversion functions and their general descriptions:

Integer Conversion (int()): Converts a value or variable to an integer data type. If the value cannot be converted to an integer (e.g., a non-numeric string), it may raise an error.

Example :

```
x = int("123")
```

Float Conversion (float()): Converts a value or variable to a floating-point data type. This is useful when you want to work with decimal numbers.

Example:

```
y = float("3.14")
```

String Conversion (str()): Converts a value or variable to a string data type. This is commonly used to concatenate non-string values into strings for display or storage.

Example:

```
z = str(42)
```

Boolean Conversion (bool()): Converts a value or variable to a boolean data type. In many programming languages, values like 0, empty strings, and None are considered "false," while other values are "true."

Example:

```
flag = bool(1)  # flag is True
```

Automatic Type Conversion (Implicit Type Conversion)

Some programming languages perform automatic type conversion when operations are performed between different data types. This is often referred to as type coercion.

Example:

result = 3 + 2.5 # Implicit conversion to float

It's important to use type conversion functions carefully to avoid unexpected errors or loss of data precision. Always ensure that the conversion is meaningful and safe for your specific use case.

V. Mathematical Functions

Python provides a rich set of mathematical functions and operations through the built-in math module. You need to import the math module to access these functions. Here are some commonly used mathematical functions and operations available in Python's math module:

Basic Mathematical Operations:

math.sqrt(x): Returns the square root of x.
math.pow(x, y): Returns x raised to the power of y.
math.exp(x): Returns the exponential value of x (e^x).
math.log(x): Returns the natural logarithm of x.
math.log10(x): Returns the base-10 logarithm of x.

Trigonometric Functions:

math.sin(x): Returns the sine of x in radians.
math.cos(x): Returns the cosine of x in radians.
math.tan(x): Returns the tangent of x in radians.
math.asin(x): Returns the arcsine (inverse sine) of x in radians.
math.acos(x): Returns the arccosine (inverse cosine) of x in radians.
math.atan(x): Returns the arctangent (inverse tangent) of x in radians.
math.atan2(y, x): Returns the arctangent of the quotient y / x with the correct sign.

Rounding Functions:

math.ceil(x): Returns the smallest integer greater than or equal to x.
math.floor(x): Returns the largest integer less than or equal to x.
math.trunc(x): Returns the integer part of x.

Constants:

math.pi: A constant representing the mathematical constant π (pi).
math.e: A constant representing the mathematical constant e (Euler's number).

Here's an example of how to use some of these mathematical functions in Python:

```
import math

x = 16
y = 2.5

# Basic mathematical operations
square_root = math.sqrt(x)
power = math.pow(x, y)
exponential = math.exp(x)
natural_log = math.log(x)
base_10_log = math.log10(x)

# Trigonometric functions
sin_x = math.sin(math.pi / 6)
cos_x = math.cos(math.pi / 3)
tan_x = math.tan(math.pi / 4)

# Rounding functions
ceil_result = math.ceil(y)
floor_result = math.floor(y)

print("Square root:", square_root)
print("Power:", power)
print("Exponential:", exponential)
print("Natural Logarithm:", natural_log)
print("Base-10 Logarithm:", base_10_log)
print("Sine:", sin_x)
print("Cosine:", cos_x)
print("Tangent:", tan_x)
print("Ceiling:", ceil_result)
print("Floor:", floor_result)
```

Remember to import the math module before using these functions. You can explore more mathematical functions and constants provided by the math module in Python's official documentation.

VII. Lambda Functions

What is a Lambda Function?

A lambda function in Python is a small, anonymous function that is defined using the lambda keyword. Unlike regular functions that are defined using the def keyword and given a name, lambda functions are typically used for short operations and don't require a name. They can take any number of arguments but only have one expression.

Syntax of a Lambda Function

lambda arguments: expression

- arguments: The inputs to the function (like the parameters in a normal function).
- expression: The single expression that is evaluated and returned.

Example 1: Simple Arithmetic Operation

Let's start with a simple example that adds two numbers.

```
add = lambda x, y: x + y
```

Explanation:

- lambda x, y: defines a lambda function that takes two arguments, x and y.
- x + y is the expression that gets evaluated and returned.

You can use this lambda function just like a regular function:

```
result = add(3, 5)
print(result)  # Output: 8
```

Example 2: Sorting a List of Tuples

Lambda functions are very useful when you need to specify a short function as an argument to other functions. For example, when sorting a list of tuples.

```
data = [(1, 'banana'), (3, 'apple'), (2, 'orange')]
sorted_data = sorted(data, key=lambda item: item[1])
```

Explanation:

- The sorted() function is used to sort the list data.
- The key argument specifies a function that extracts the part of the data that should be used for sorting.
- lambda item: item[1] defines a lambda function that takes a tuple item and returns the second element (item[1]), which is the string (fruit name) in this case.

This sorts the list of tuples by the fruit names:

```
print(sorted_data)  # Output: [(3, 'apple'), (1, 'banana'), (2, 'orange')]
```

Example 3: Filtering a List

You can use lambda functions with the filter() function to filter elements in a list.

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
```

Explanation:

- filter() applies a function to each item in the list and returns only those items for which the function returns True.
- lambda x: x % 2 == 0 is a lambda function that returns True if x is even.

This filters out only the even numbers from the list:

```
print(even_numbers)  # Output: [2, 4, 6, 8, 10]
```

Example 4: Mapping Values in a List

Lambda functions can be used with the map() function to apply a transformation to each item in a list.

```
numbers = [1, 2, 3, 4, 5]
squared_numbers = list(map(lambda x: x ** 2, numbers))
```

Explanation:

- map() applies the given function to each item in the list and returns a new list with the results.
- lambda x: x ** 2 is a lambda function that squares x.

This returns a new list where each number is squared:

```
print(squared_numbers)  # Output: [1, 4, 9, 16, 25]
```

Example 5: Conditional Expressions in Lambda

You can use conditional expressions within lambda functions, similar to a ternary operator.

```
max_value = lambda x, y: x if x > y else y
```

Explanation:

- lambda x, y: defines a lambda function with two arguments.
- x if x > y else y is a conditional expression that returns x if x is greater than y, otherwise it returns y.

This lambda function returns the maximum of two values:

```
result = max_value(10, 20)
print(result)  # Output: 20
```

Summary

Lambda functions are a concise way to define small, one-time-use functions. They're particularly useful when you need to pass a simple function as an argument to another function, like in sorting, filtering, or mapping operations. However, for more complex logic, it's better to define a regular function using the def keyword.

Exercise chatGPT

Use GPT to write a program:

Help me develop a simple python program. I want to calculate the total electricity bill. The rules are as follows: Installed load: The installed load is the maximum amount of electricity that your home or business can use at any given time. This is measured in kilowatts (kW). The higher your installed load, the higher your fixed charges will be.Energy consumption: The energy consumption is the total amount of electricity that you use in a billing period. This is measured in units (kWh). The more electricity you consume, the higher your variable charges will be.Tariff slabs: The tariff slabs are the different rates per unit of electricity that you are charged. There are currently 4 tariff slabs in Bangalore:

Up to 50 units: Rs 4.75/unit
51 to 100 units: Rs 5.75/unit

101 to 200 units: Rs 7/unit
Above 200 units: Rs 8/unit

The actual electricity charges that you pay will depend on your installed load, energy consumption, and the tariff slab that you fall into.

After the ILT, the student has to do a 30 min live lab. Please refer to the Day 5 Lab doc in the LMS.